

Danmarks
Tekniske
Universitet



**42136 - Large Scale Optimization using Decomposition
Assignment Part I**

AUTHORS

Emmanuel Denzel Nana Kofi Dodoo - s225101
Harjas Preet - s215168

March 18, 2026

Contents

1	Benders Decomposition	1
1.1	Mathematical Formulation	1
1.1.1	Decision Variables	1
1.1.2	Objective Function	1
1.1.3	Constraints	1
1.2	Solution of the Model	2
1.3	Benders Decomposition Reformulation	2
1.3.1	Master Problem	3
1.3.2	Scenario Subproblem	3
1.3.3	Dual Subproblem	3
1.3.4	Benders Optimality Cuts	4
1.4	Benders Decomposition Implementation	4
1.5	Benders Decomposition with Feasibility Cuts	5
1.6	Single cut vs. Multicut	5
2	Progressive Hedging	6
2.1	The Core Idea	6
2.2	PH Scenario Subproblem Formulation	7
2.3	Application and Results	9

1 Benders Decomposition

1.1 Mathematical Formulation

The company must determine the capacity to reserve from a set of potential suppliers before the client's demand is recognized. Demand uncertainty is represented by a finite set of scenarios, each associated with a probability of occurrence. Once a scenario is known, the reserved supplier capacity can be used to serve client demands. If the available capacity is insufficient to fully satisfy demand, the unmet portion is penalised. The objective is therefore to determine a capacity reservation plan that minimizes the expected total cost.

1.1.1 Decision Variables

First-stage variables

- $y_i \in \{0, 1\}$: equals 1 if supplier i is selected, 0 otherwise
- $x_i \geq 0$: capacity reserved from supplier i

Second-stage variables (scenario dependent)

- $z_{ijs} \geq 0$: units shipped from supplier i to client j in scenario s
- $w_{js} \geq 0$: unmet demand of client j in scenario s

1.1.2 Objective Function

The objective is to minimize the expected total cost consisting of fixed supplier reservation costs, capacity reservation costs, operating costs, and penalties for unmet demand.

$$\min \sum_{i \in I} f_i y_i + \sum_{i \in I} c_i x_i + \sum_{s \in S} p_s \left(\sum_{i \in I} \sum_{j \in J} t_{ij} z_{ijs} + \sum_{j \in J} u w_{js} \right) \quad (1)$$

1.1.3 Constraints

Supplier capacity limits

$$x_i \leq b_i y_i \quad \forall i \in I \quad (2)$$

Limit on the number of suppliers

$$\sum_{i \in I} y_i \leq n \quad (3)$$

Capacity usage constraints

$$\sum_{j \in J} z_{ijs} \leq x_i \quad \forall i \in I, s \in S \quad (4)$$

Demand satisfaction

$$\sum_{i \in I} z_{ijs} + w_{js} = d_{js} \quad \forall j \in J, s \in S \quad (5)$$

Variable domains

$$y_i \in \{0, 1\} \quad \forall i \in I \quad (6)$$

$$x_i \geq 0 \quad \forall i \in I \quad (7)$$

$$z_{ijs} \geq 0 \quad \forall i \in I, j \in J, s \in S \quad (8)$$

$$w_{js} \geq 0 \quad \forall j \in J, s \in S \quad (9)$$

1.2 Solution of the Model

The mathematical model presented in the previous section was implemented in Julia. The optimal objective value obtained from the solver is **11922.8**. The optimal solution selects seven suppliers, which corresponds to the maximum number of suppliers allowed by the constraint $\sum_{i \in I} y_i \leq n$. The selected suppliers and their corresponding reserved capacities are presented in Table 2.

Supplier i	Reserved Capacity x_i
4	82
5	300
8	90
9	235
11	430
12	385
13	433

Table 1: Selected suppliers and reserved capacities in the optimal solution

1.3 Benders Decomposition Reformulation

The problem exhibits a natural two-stage structure. The first-stage decisions determine which suppliers are selected and how much capacity is reserved before the uncertain demand

is known. Once a scenario is realized, second-stage decisions determine how the reserved capacity is allocated to clients and how much demand remains unmet.

1.3.1 Master Problem

The Benders master problem determines the first-stage decisions and an approximation of the expected second-stage cost.

$$\min \quad \sum_{i \in I} f_i y_i + \sum_{i \in I} c_i x_i + \theta \quad (10)$$

subject to

$$x_i \leq b_i y_i \quad \forall i \in I \quad (11)$$

$$\sum_{i \in I} y_i \leq n \quad (12)$$

$$\theta \geq \text{Benders cuts} \quad (13)$$

$$y_i \in \{0, 1\}, \quad x_i \geq 0 \quad (14)$$

1.3.2 Scenario Subproblem

For a fixed first-stage solution x obtained from the master problem, a second-stage subproblem is solved for each scenario $s \in S$:

$$\min \quad \sum_{i \in I} \sum_{j \in J} t_{ij} z_{ijs} + \sum_{j \in J} u w_{js} \quad (15)$$

subject to

$$\sum_{j \in J} z_{ijs} \leq x_i \quad \forall i \in I \quad (16)$$

$$\sum_{i \in I} z_{ijs} + w_{js} = d_{js} \quad \forall j \in J \quad (17)$$

$$z_{ijs}, w_{js} \geq 0 \quad (18)$$

1.3.3 Dual Subproblem

The dual variables:

- π_{is} for capacity constraints
- α_{js} for demand constraints

Using the dual variables, the dual of the scenario subproblem can be written as

$$\max \quad \sum_{j \in J} d_{js} \alpha_{js} + \sum_{i \in I} x_i \pi_{is} \quad (19)$$

subject to

$$\alpha_{js} + \pi_{is} \leq t_{ij} \quad \forall i \in I, j \in J \quad (20)$$

$$\alpha_{js} \leq u \quad \forall j \in J \quad (21)$$

$$\pi_{is} \leq 0 \quad (22)$$

1.3.4 Benders Optimality Cuts

Optimal solutions of the dual subproblem generate Benders optimality cuts:

$$\theta \geq \sum_{s \in S} p_s \left(\sum_j d_{js} \alpha_{js} + \sum_i x_i \pi_{is} \right) \quad (23)$$

The cuts approximate the true recourse function. Because the unmet demand is penalised by w_{js} , the subproblem is always feasible. Therefore, only optimality cuts are required in the Benders decomposition.

1.4 Benders Decomposition Implementation

The Benders decomposition algorithm was implemented in Julia using the JuMP optimization framework and the HiGHS solver. A single-cut Benders approach was used, where a single optimality cut is added to the master problem at each iteration.

The algorithm proceeds iteratively by solving the master problem to obtain first-stage decisions, followed by solving scenario-specific dual subproblems. The lower bound (LB) is given by the master problem objective value, while the upper bound (UB) is computed by evaluating the first-stage solution in the subproblems and calculating the corresponding expected recourse cost.

Optimality cuts are derived from the dual solutions of the subproblems and added to the master problem. These cuts provide increasingly accurate approximations of the expected second-stage cost. Since unmet demand is allowed through penalty variables, the subproblems are always feasible, and therefore only optimality cuts are required.

The algorithm terminates when the optimality gap satisfies the convergence criterion

$$UB - LB \leq \epsilon.$$

Using this implementation, the algorithm converged after **100 iterations**. The final objective value obtained was **11922.8**, which matches the optimal value obtained in Section 1.2. This confirms the correctness of the Benders decomposition approach.

1.5 Benders Decomposition with Feasibility Cuts

In this part, we assume that all client demand must be satisfied, meaning that the unmet demand variables w_{js} are removed from the model. As a result, the demand satisfaction constraint becomes:

$$\sum_{i \in I} z_{ijs} = d_{js} \quad \forall j \in J, s \in S \quad (24)$$

This constraint enforces that all demand must be satisfied using the reserved supplier capacity. In this setting, the second-stage subproblem may become infeasible if the capacity reserved in the first stage is insufficient to satisfy the demand in a given scenario.

In Benders decomposition, an infeasible primal subproblem corresponds to an unbounded dual subproblem. The unbounded direction of the dual problem defines an extreme ray, which can be used to generate a feasibility cut. These feasibility cuts are then added to the master problem to restrict the feasible region and ensure that future first-stage solutions reserve sufficient capacity.

In the initial iterations, infeasible subproblems were observed for several scenarios due to insufficient reserved capacity. Consequently, feasibility cuts were generated and added to the master problem. After adding these cuts, the master problem was re-optimized, and the algorithm continued iteratively. The Benders decomposition algorithm converged after **7 iterations**, yielding a final objective value of **13609.55**. The resulting objective value is higher than in the previous case, reflecting the increased restrictiveness of the model when unmet demand is not allowed.

1.6 Single cut vs. Multicut

We implement two variants of Benders decomposition: a single-cut and a multi-cut version. The difference between the two approaches lies in how the optimality cuts are generated and incorporated into the master problem.

As illustrated in Figure 1, the performance gap between the two variants is substantial. The multi-cut approach reached convergence in only 7 iterations, with the lower bound jumping significantly in the very first steps. This behavior can be explained by the fact that the master problem receives one constraint per scenario in each iteration, providing a more detailed representation of how different scenarios contribute to the recourse cost and potential infeasibilities.

On the other hand, the single-cut version was much slower, taking over 100 iterations. Since

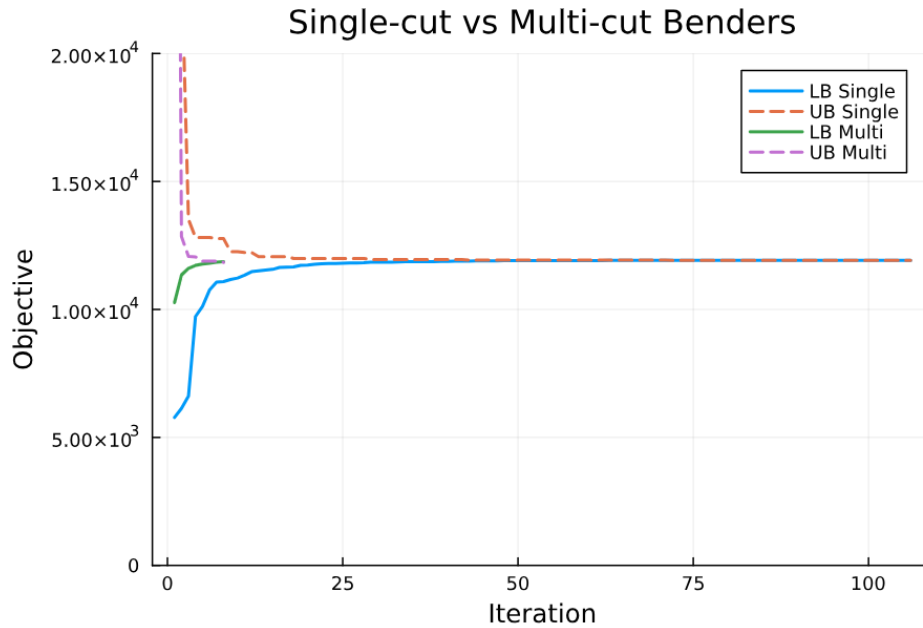


Figure 1: Evolution of the lower bound (LB) and upper bound (UB) for the single-cut and multi-cut Benders decomposition methods.

it averages the dual information into one aggregate cut, the master problem has a coarser approximation of the recourse function and ends up making smaller, more timid adjustments to the capacity variables x_i at each step. While the single-cut master problem is technically faster to solve because it has fewer constraints, the massive reduction in total iterations makes multi-cut the clear winner for this specific dataset.

2 Progressive Hedging

2.1 The Core Idea

Progressive Hedging algorithms solve two-stage stochastic optimization problems by decomposing them into smaller scenario-based subproblems. In a two-stage stochastic problem, the first-stage decisions must be made before any uncertainty is realised. Uncertainties could be future demand levels, potential disruptions, supply availability, etc. Second-stage decisions come after the first stage and they are made after the scenario is known.

The issue with solving these problems is the fact that the first stage variables must be identical across all scenarios which ends up coupling scenarios together into one large optimization problem. This is where the Progressive Hedging algorithm comes in. It temporarily relaxes this coupling so that each scenario can be solved independently. It creates scenario-specific copies of the first-stage variables so instead of using the single variable x_i , it introduces x_{is} , which represents the first-stage decision for supplier (i) in scenario (s). This allows each scenario to be solved as an independent optimization problem with its own first- and

second-stage decisions and scenario-specific constraints.

We must remember though, in the original stochastic problem, the first-stage decisions must be the same for all scenarios. This requirement is called 'non-anticipativity'. It describes how first-stage decisions must be made before uncertain outcomes are revealed, meaning the decision maker does not yet know which scenario will occur. Therefore, the capacity decision must be based only on the information available at that time and cannot depend on the scenario that eventually occurs. Instead of enforcing these equality constraints directly, the PH algorithm enforces non-anticipativity through an iterative update process. After solving all scenario subproblems, a consensus value for the first-stage variables is computed. This consensus capacity is given by the probability-weighted average across scenarios,

$$\bar{x}_i = \sum_{s \in S} p_s x_{is}$$

Penalty terms are then added to each scenario objective that penalize deviations from this consensus value. These penalties gradually push the scenario-specific decisions toward agreement. In each iteration the scenario problems are solved again, the consensus value is updated, and the penalties are adjusted. As the algorithm progresses, the scenario-specific first-stage variables converge to the same value, satisfying the non-anticipativity requirement. This approach allows the large stochastic optimization problem to be solved through smaller scenario-based problems.

2.2 PH Scenario Subproblem Formulation

In Progressive Hedging framework, the stochastic capacity reservation problem is decomposed into scenario-specific subproblems. Each scenario $s \in S$ is solved independently with its own copy of the first-stage decision variable.

The first-stage capacity decision is represented by the variable

$$x_{is}$$

which denotes the capacity reserved from supplier i in scenario s .

The second-stage allocation decisions are represented by the variables

$$q_{ijs}, \quad z_{js}$$

where q_{ijs} is the amount shipped from supplier i to client j in scenario s , and z_{js} represents the unmet demand for client j in scenario s .

For each scenario s , the PH subproblem is therefore defined with decision variables x_{is} , q_{ijs} , and z_{js} , subject to the scenario constraints

$$\begin{aligned}\sum_{j \in J} q_{ijs} &\leq x_{is} \quad \forall i \in I, \\ \sum_{i \in I} q_{ijs} + z_{js} &= d_{js} \quad \forall j \in J.\end{aligned}$$

The objective of each scenario subproblem minimizes the scenario-specific cost

$$\min \sum_{i \in I} c_i x_{is} + \sum_{i \in I} \sum_{j \in J} t_{ijs} q_{ijs} + \sum_{j \in J} u_{js} z_{js}.$$

To enforce non-anticipativity of the first-stage capacity decisions, the PH algorithm computes a consensus capacity vector after each iteration. This consensus value is given by the probability-weighted average of the scenario-specific capacities,

$$\bar{x}_i = \sum_{s \in S} p_s x_{is}.$$

The scenario subproblems are then updated using this consensus value, and the process is repeated until the scenario-specific first-stage decisions converge. In order to enforce non-anticipativity of the first-stage decisions, the Progressive Hedging algorithm augments the scenario subproblem objective with penalty and multiplier terms that penalize deviations from the consensus capacity value. This allows the scenario problems to be solved independently while gradually driving the first-stage decisions toward a common value.

For each scenario $s \in S$, the augmented PH subproblem can therefore be written as

$$\min \sum_{i \in I} c_i x_{is} + \sum_{i \in I} \sum_{j \in J} t_{ij} q_{ijs} + \sum_{j \in J} u z_{js} + \sum_{i \in I} \lambda_{is} (x_{is} - \bar{x}_i) + \frac{\rho}{2} \sum_{i \in I} (x_{is} - \bar{x}_i)^2$$

subject to the scenario constraints

$$\begin{aligned}\sum_{j \in J} q_{ijs} &\leq x_{is} \quad \forall i \in I, \\ \sum_{i \in I} q_{ijs} + z_{js} &= d_{js} \quad \forall j \in J, \\ 0 &\leq x_{is} \leq B_i \quad \forall i \in I.\end{aligned}$$

After solving the scenario subproblems, the Progressive Hedging algorithm computes an updated consensus value for the first-stage capacity decisions. This consensus value is calculated as the probability-weighted average across all scenarios,

$$\bar{x}_i^{k+1} = \sum_{s \in S} p_s x_{is}^{k+1}.$$

The multipliers are then updated to penalize remaining deviations from the consensus solu-

tion. The multiplier update rule is given by

$$\lambda_{is}^{k+1} = \lambda_{is}^k + \rho(x_{is}^{k+1} - \bar{x}_i^{k+1}).$$

These updates are repeated iteratively. As the algorithm progresses, the scenario-specific first-stage capacity decisions gradually move toward the same value, ensuring that the non-anticipativity requirement is satisfied while still allowing the problem to be solved through smaller scenario-based subproblems.

2.3 Application and Results

The algorithm for this question was implemented in Julia using JuMP and Gurobi solver. The algorithm converged in 57 iterations, achieving a final residual of 5.87×10^{-3} . The optimal capacity reservation vector $\bar{x}$ is given in Table 2.

Supplier	$\bar{x}_i$
1	55.31
2	0.00
3	0.00
4	117.01
5	311.18
6	140.99
7	0.00
8	81.77
9	75.61
10	0.00
11	430.00
12	310.05
13	433.00

Table 2: Optimal capacity reservation decisions

The corresponding expected objective value is 11458.63.

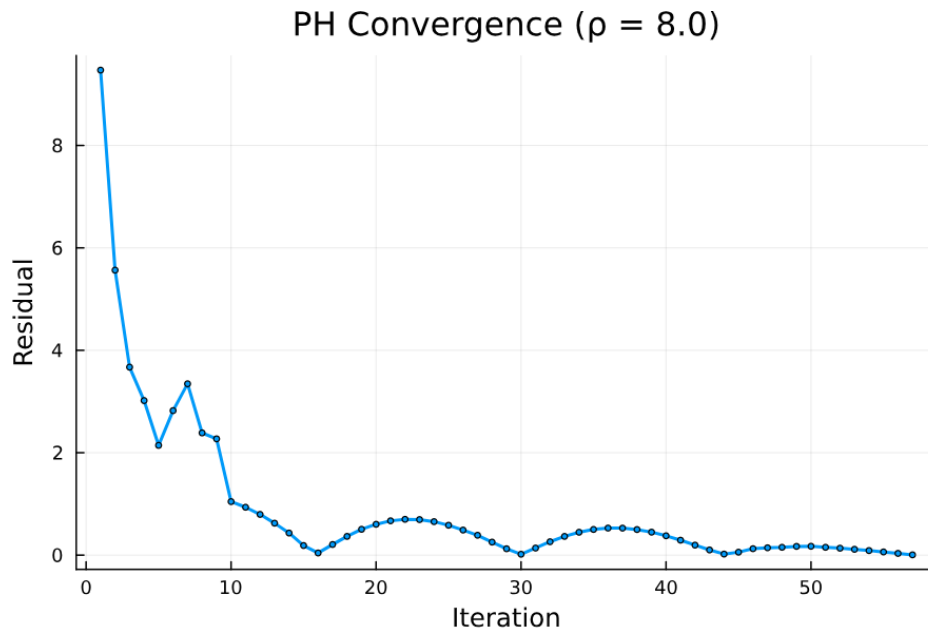


Figure 2: Convergence Plot

The convergence behaviour is shown in Figure 2. The residual decreases quickly in the first few iterations, showing that the scenario solutions are rapidly trying to move toward agreement. Some small fluctuations appear in the middle iterations, which is normal in Progressive Hedging because of the updates of the penalty and dual variables. The deviations from the consensus are penalized through the dual variables and these can cause scenario solutions to overshoot in future iterations. This results in temporary oscillations as the algorithm alternates between over- and under-correction before converging. After about 10 iterations, the residual decreases more steadily until it converges.